

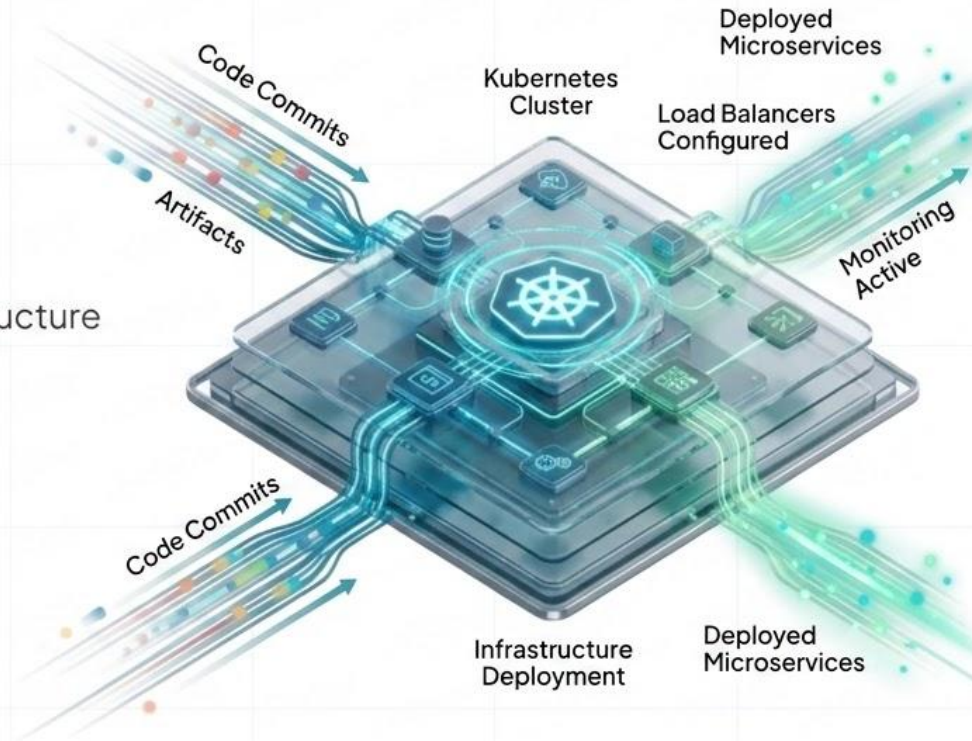
DevOps Engineer Bootcamp Final Project

Core Banking Syariah (CBS)

Micro Service Architecture & Automated Infrastructure
Deployment on Kubernetes

Renaldy Imran Hermawan

Junior DevOps Engineer



Background & Scope: CBS Project

Project Description

Web-based Islamic digital banking simulation designed with a modern microservices architecture comprising 2 frontend portals and 4 backend services.

Strategic Challenges

- ✓ Managing polyglot deployments
- ✓ Cross-service communication
- ✓ Maintaining data consistency

Microservices Stack Portfolio

Admin Portal
portal-admin
VueJS Framework

Customer Portal
portal-mobile
ReactJS Mobile

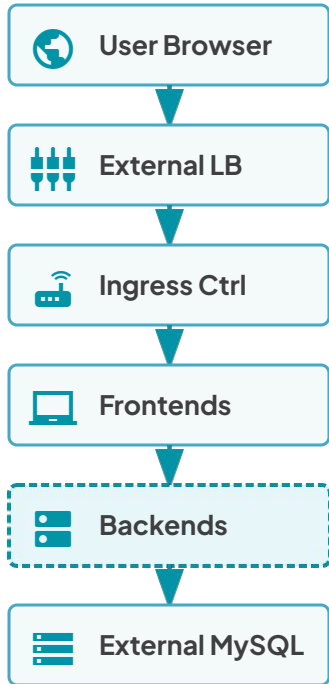
Auth Service
api-auth
Golang Runtime

Customer API
api-customer
Node.js / JS

Transactions
api-transaction
Java Enterprise

Financing
api-financing
Golang Runtime

Detailed System Architecture & Network Diagram



Ingress to Frontend Flow

External requests originate from the **User Browser**, routed through a high-availability **External Load Balancer**. The **Ingress Controller** acts as the gateway, decrypting SSL and mapping requests to the appropriate **Frontend Portals** based on host/path rules.

Internal Service Mesh & Routing

Communication between Frontends and the internal business logic (**Backends**) is strictly managed via **Kubernetes Cluster-IP**. This ensures that internal traffic never leaves the private virtual network, providing low-latency and secure service discovery.

Persistent Data Layer: Single Source of Truth

The persistent state is maintained in a centralized **External MySQL** instance. This serves as the authoritative single source of truth for all banking operations, ensuring data consistency across the cluster.

Endpoint: 103.160.62.159:3308

Schema: core_banking_syariah

Protocol: TCP/IP (Standard MySQL Connection)

Customized GitOps & Kustomize Repository Strategy

GitLab Subgroup: <https://gitlab.com/tutorial-ci-dibimbing/renaldy-imran-cbs-final-projek>



Kustomize Architecture

Clean separation of **'base'** configurations and environment-specific **'overlays'** (dev, uat, prod) for DRY manifest management.



Automated Parameterization

Dynamic container **image tag updates** and **environment variables** injected via custom GitOps pipeline workflows.

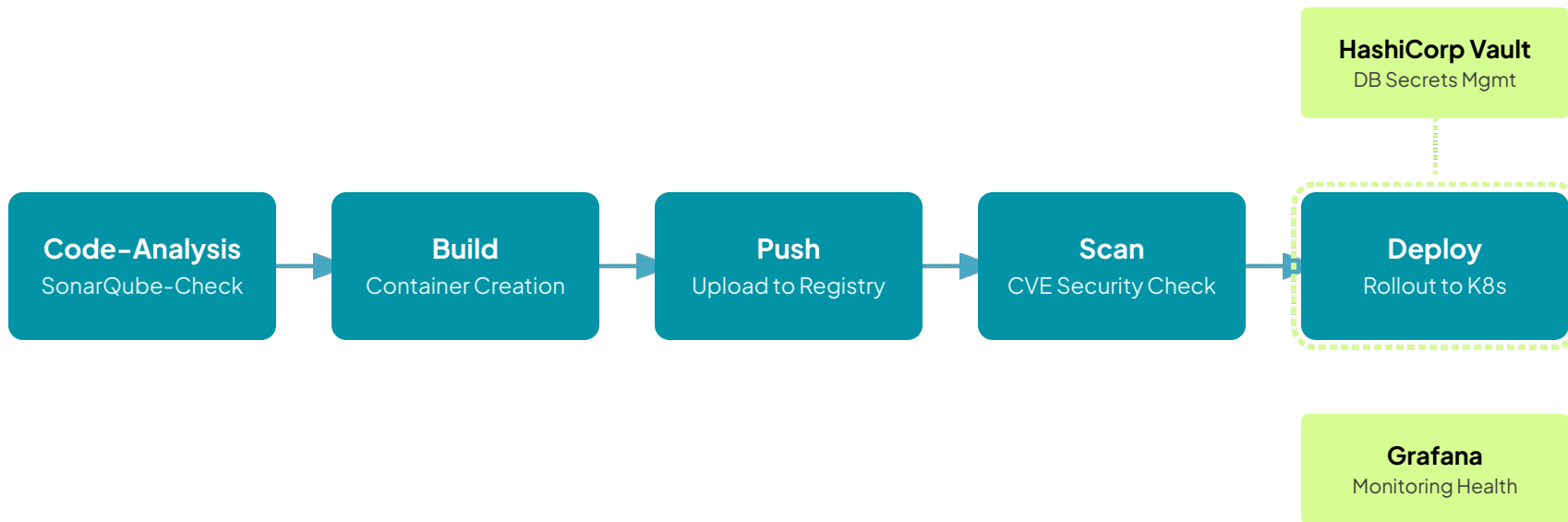


Multi-Environment Alignment

Git branches mapped directly to Kustomize overlays:

```
dev -> overlays/dev  
uat -> overlays/uat  
prod -> overlays/prod
```

Automated CI/CD Pipeline Workflow



The pipeline orchestrates a seamless flow from code commit to production, with **Vault** ensuring secure database credential injection and **Grafana** providing real-time deployment health metrics during the Kubernetes rollout.

GitOps Repository Directory Structure & Kustomize Layout

Declarative Multi-Environment Manifest Organization

```
gitops-manifests-repo/
├── apps/
├── api-auth / api-customer / api-financing ...
├── base/ (deployment, service, kustomization)
├── overlays/
├── dev / uat/ (kustomization.yaml)
├── prod/ (hpa, vso-secrets, kustomization)
├── portal-admin / portal-mobile
├── base/ (deployment, service, kustomization)
├── overlays/
├── dev / uat/ (ingress, kustomization)
├── prod/ (ingress, hpa, vso-secrets)
├── namespaces/
├── dev.yaml
├── uat.yaml
└── prod.yaml
```



Apps Hierarchy

Isolated base & overlay configurations for all 6 core microservices.



Production Security & Scaling

Prod overlays integrate HPA autoscaling & Vault Secret Operator secrets.



Ingress Routing

Frontend portals define environment-specific Ingress rules in overlays.



Environment Isolation

Dedicated namespace manifests defining cluster boundaries safely.

Kubernetes Runtime & Deployment Strategy



Isolation

Dedicated namespaces for environment separation:

- `renaldy-cbs-dev`
- `renaldy-cbs-uat`
- `renaldy-cbs-prod`



Governance

Strict resource management and safety tiers:

- ResourceQuotas enforced per namespace
- Mandatory Pod Requests/Limits for CPU and RAM



Scaling Strategy

DEV / UAT Environments

Fixed static deployment: 1 replica per service.

PRODUCTION HIGHLIGHT

Horizontal Pod Autoscaler (HPA)

Dynamic scaling between **2 – 5 replicas** based on resource utilization.

Monitoring & Observability with Grafana

Target Dashboard: renaldy-imran-cbs-dev

Target Service: api-auth

Tracked Metrics

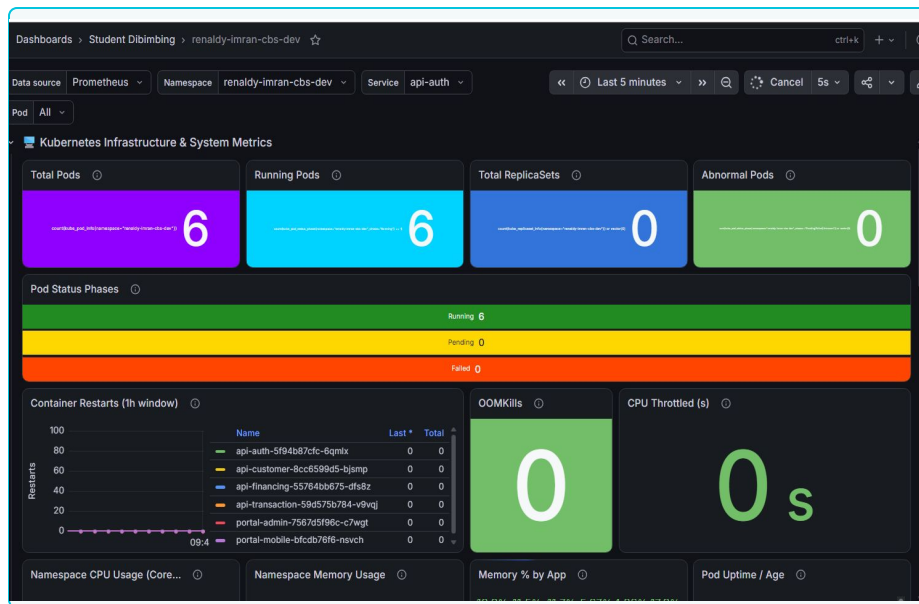
Request Rate (RPS)

Error Rates (4xx/5xx)

p95/p99 Latency

Resource Usage (CPU/RAM)

Pod Healthchecks



Verification Testing & User Access Roles

ROLE	CREDENTIALS	ACCESS LEVEL
 Admin Role	Username: admin Password: Admin123!	Full management of customers, balances, and reports.
 Customer Role	Usernames: nasabah001 - 010 Passwords: Nasabah001! - 010!	Readonly access to Mobile Portal for balances and history.

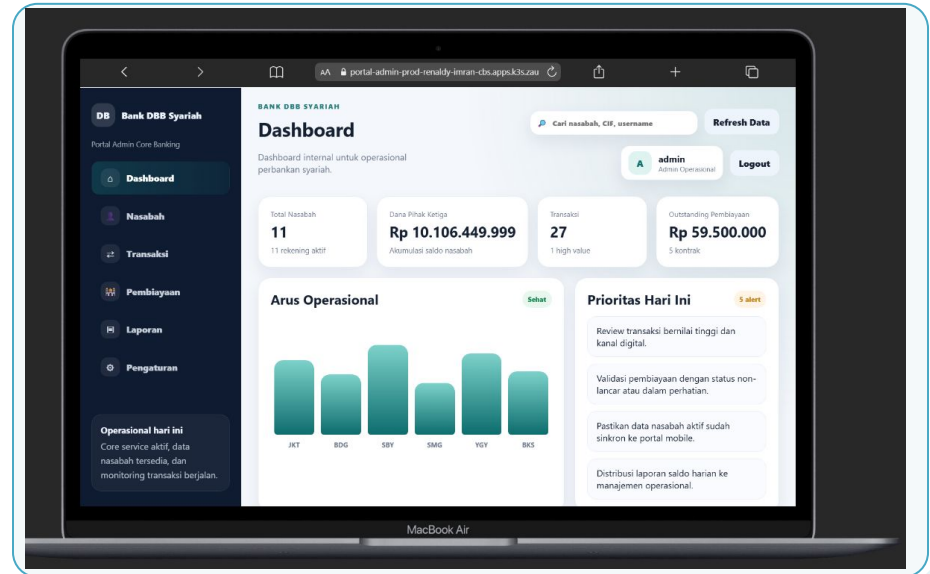
Application Interface: Bank DBB Syariah

Portal Admin

Enterprise-grade management interface for banking operations.

Technology Stack

VueJS



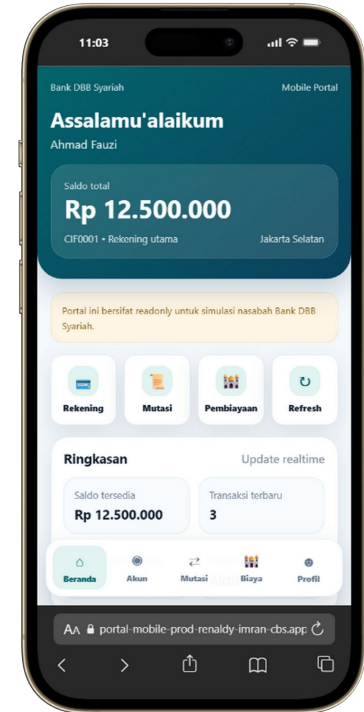
Preview: Admin Portal Dashboard Interface

Application Interface: Bank DBB Syariah – Portal Mobile

Tech Stack: ReactJS

Key Features

- Readonly customer view for secure information access
- Real-time balance checks and immediate updates
- Transaction history ledger with detailed search
- Financing status tracking and payment schedules



Conclusion & Lessons Learned



Achievements

Successful automated deployment of 6 polyglot microservices on Kubernetes.

Infrastructure Stabilized



Technical Insights

- Container optimization strategies
- HashiCorp Vault for secrets management
- Grafana for real-time observability



Future Roadmap

- Implementation of Istio Service Mesh
- Automated DB disaster recovery systems